

Linear fit simple

```
A=vstack([x,[1]*w.nrows]).T
a,b=linalg.lstsq(A,y)[0]
```

```
+-----+
A = | 0.1 | 1.0 |
    | 0.5 | 1.0 |
    | 1.0 | 1.0 |
    | 2.5 | 1.0 |
    | 5.0 | 1.0 |
    | 10.0 | 1.0 |
    | 25.0 | 1.0 |
    | 50.0 | 1.0 |
    +-----+
```

Linear fit through zero

```
A=vstack([x,[0]*w.nrows]).T
a,b=linalg.lstsq(A,y)[0]
```

```
+-----+
A = | 0.1 | 0.0 |
    | 0.5 | 0.0 |
    | 1.0 | 0.0 |
    | 2.5 | 0.0 |
    | 5.0 | 0.0 |
    | 10.0 | 0.0 |
    | 25.0 | 0.0 |
    | 50.0 | 0.0 |
    +-----+
```

Quadratic fit simple

```
A=vstack([x,[1]*w.nrows,array(x)**2]).T
a,b,c=linalg.lstsq(A,y)[0]
```

```
+-----+
A = | 0.1 | 1.0 | 0.01 |
    | 0.5 | 1.0 | 0.25 |
    | 1.0 | 1.0 | 1.0 |
    | 2.5 | 1.0 | 6.25 |
    | 5.0 | 1.0 | 25.0 |
    | 10.0 | 1.0 | 100.0 |
    | 25.0 | 1.0 | 625.0 |
    | 50.0 | 1.0 | 2500.0 |
    +-----+
```

Quadratic fit through zero

```
A=vstack([x,array(x)**2,[0]*w.nrows]).T
a,b,c=linalg.lstsq(A,y)[0]
```

```
+-----+
A = | 0.1 | 0.01 | 0.0 |
    | 0.5 | 0.25 | 0.0 |
    | 1.0 | 1.0 | 0.0 |
    | 2.5 | 6.25 | 0.0 |
    | 5.0 | 25.0 | 0.0 |
    | 10.0 | 100.0 | 0.0 |
    | 25.0 | 625.0 | 0.0 |
    | 50.0 | 2500.0 | 0.0 |
    +-----+
```

Probably, the simplest way is the use of *polyfit* with the syntax *coef=polyfit(x,y,degree)*. For a better graphical presentation, I would like to say something about the use of LaTeX and about the couple Pandas + Seaborn, specifying also that the Pandas and Seaborn packages have more complex applications than the one presented here. LaTeX must be previously installed on your system; then just add *rc("text",usetex=True)* in your script. The result is shown in Figure 2. About Pandas, in the following example, a *DataFrame* with three columns is created. Then, *x* vs *y* data are plotted using Seaborn with *regplot* (*lw=0* and *marker="o"*) and, last, the linear model is plotted always with *regplot* but with some different options (*lw=1* and *marker=""*). The result is shown in Figure 3, which is practically the same as with *ggplot* for R. Note that the *ggplot* plotting system exists also for Python and it is available at <http://ggplot.yhathq.com>. Another way to obtain a *ggplot* like plot is with the use of a style sheet. For example, put *style.use("ggplot")* before the plot command. To know all the styles available just use *print(style.available)*.

```
import pandas as p
import seaborn as s
coef=polyfit(x,y,1)
yp=polyval(coef,x)
df=p.DataFrame({"Conc":x,"Abs":y,"Abs pred":yp})
s.regplot(x="Conc",
          y="Abs",
          data=df,
          ci=False,
          scatter_kws={"color":"g","alpha":0.5,"s":90},
          line_kws={"color":"w","alpha":0,"lw":0},
          marker="o")
s.regplot(x="Conc",
          y="Abs pred",
          data=df,
          ci=False,
          scatter_kws={"color":"w","alpha":0,"s":0},
          line_kws={"color":"k","alpha":1,"lw":1},
          marker="")
```

The data frame here is printed via *prettytable*:

```
+-----+
| Conc | Abs | Abs pred |
+-----+
| 0.05 | 0.046 | 0.053 |
| 0.10 | 0.093 | 0.092 |
| 0.20 | 0.171 | 0.172 |
| 0.25 | 0.217 | 0.212 |
| 0.50 | 0.418 | 0.412 |
| 1.00 | 0.807 | 0.811 |
+-----+
```